Zero Breach Toolkit

Security hardening and engineering remediation specification

Implementation-ready technical requirements | Version 1.0 | 29 October 2026

Basis: uploaded Zero-Breach-Toolkit-main.zip and static security audit dated 29 October 2026; repository code identifies version 0.1.11. This specification proposes changes; it does not certify that they have been implemented.

DOCUMENT STATUS: TECHNICAL DESIGN BASELINE — SUBJECT TO IMPLEMENTATION, CODE REVIEW AND TEST VALIDATION

Executive summary — one page

The toolkit has a read-only, artefact- and indicator-based scanning design, but it handles highly sensitive data and includes optional networked updates, a browser-based online test and execution of a separate Android collector. The previous static assessment found no obvious deliberate backdoor in the inspected scanning path; absence of observed malicious logic is not proof of safety. The objective of this specification is to make the trust boundaries enforceable, minimise sensitive-data exposure and demonstrate predictable, testable behaviour across supported platforms.

Release decision

Do not label the toolkit “safe”, “offline” or “malware-free” without qualifications. Designate normal scanning OFFLINE BY DEFAULT, read-only and least-privilege; keep Android acquisition, downloads and external browser actions disabled unless separately consented to. For deployment on sensitive devices, require operating-system-enforced outbound isolation and verified source provenance. The Android collector must remain disabled until independently attested and tested.

Order	Requirement	Release implication
P0	Explicit command isolation; no-privilege elevation; secure report/evidence lifecycle; bounded parsers; authenticated external collector	Required before recommended routine use
P1	Signed supply chain and updates; robust hostile-path defences; evidence validation; fail-closed reporting	Required for security-assured release
P2	Full platform QA, fuzzing, benchmarks, detection calibration and continuous assurance	Required for mature supported product

Success measures

- Zero offline network traffic, including child processes; OS-level enforcement proved on each supported platform.
- No unsolicited elevation, downloads, persistence, external browser launches or third-party programme execution.
- Redacted default reports, recipient-encrypted full evidence, private scratch and tested interrupted-run cleanup.
- Bounded hostile-input processing; explicit completeness and error statuses; signed releases, SBOM and independent review.

Assurance limit: these controls are proposed, not implemented. The underlying audit was static, not live malware analysis or platform verification.

1. Product scope, trust model and non-goals

Primary supported deployment: Windows, macOS and Linux host-side analysis. iPhone/iPad are investigated through locally acquired backups or diagnostic artefacts, not through unrestricted native on-device scanning. Android analysis uses a distinct host-side acquisition component and must not be represented as equivalent to local Python-only scanning.

Protected assets: host/user identifiers; browsing histories and cookies; decrypted mobile backup data; private messages; collected evidence; IOC integrity; availability of the target system; user confidence in scan conclusions.

Threat actors considered: malicious dependency maintainer; compromised update server; hostile downloaded collector; untrusted crafted input/archive/database; local unprivileged adversary; local administrator/malware already present; filesystem path-swap attacker; network observer; accidental cloud synchronisation. Protection from an already compromised kernel or administrator is out of scope and must be stated.

Trust-boundary map

Boundary	Default control	Required enforcement
Host scanner → host artefacts	Read-only, least privilege	Allow-listed paths, no-follow opening, bounded reads, explicit denial audit
Scanner → network	No network	OS deny-all egress in high-assurance mode, child-process containment, packet-capture verification
Update utility → public infrastructure	Explicit update only	Separate process, verified signatures and digests, atomic staging and no scanner permission inheritance
Scanner → reports/scratch	Local only	Private per-run directory, encrypted full-fidelity evidence, redacted routine output
Android collector → device	Disabled by default	Independent signature/source attestation, user consent, sandbox, version/permission disclosure
Scanner → PowerShell/other tools	Static commands only	Absolute binaries, fixed arguments, sterile environment, hard timeout and output cap
User → results	Evidence not verdict	Coverage/error ledger; confidence wording; machine-readable schema

2. Security architecture — target design

Divide functionality into five components: (A) offline scan core with no network and no privilege-escalation API; (B) acquisition service for mobile/backups isolated from core; (C) update service that alone permits network access; (D) privacy/evidence service with explicit retention and encryption policy; (E) report/verification service using versioned schemas. Communication between components must use narrowly typed files/messages, not shared unrestricted working directories.

Recommended package split: zbtriage/core, zbtriage/platform, zbtriage/acquisition, zbtriage/updates, zbtriage/privacy, zbtriage/evidence, zbtriage/verification. This is a proposed target, not a description of present directories.

2.1 Modes and invariant policy

Mode	Network	Privileges	Output	External executables
offline (default)	Prohibited	Standard user	Redacted report	No downloaded executables
offline-high-assurance	OS-enforced prohibited	Standard user	Redacted/encrypted	Allow-listed signed local tools only
update	Allow-listed HTTPS only	Standard user	Signed feed cache	None
acquire-android	Explicitly disclosed per action	Minimum necessary	Private acquisition directory	Pre-approved attested collector
forensic-export	No network	Standard user	Recipient-encrypted bundle	None

A single immutable run-policy object must capture mode, consent, permitted roots, resource ceilings, output policy, network policy and acquisition permissions. Every module receives the relevant narrow capability; modules must not read ambient environment to silently widen privileges or network access.

3. Remediation requirements (testable specification)

SEC-001 — Hard egress containment and zero-contact IOC handling

Priority: P0 | Proposed owner: Security/platform

Required implementation: Offline scanner must never perform DNS lookups, HTTP requests, raw socket egress or outbound access via children/native code. Keep current socket guard as a defence-in-depth alarm only. High-assurance mode launches inside platform-specific deny-all network containment (e.g. tested OS firewall or isolated VM/container); refuse to claim OS enforcement when unavailable. Do not resolve indicator strings.

Verification / acceptance: On each supported OS, instrument DNS and packet capture: zero egress across clean, infected and adversarial fixtures; force a child tool to try egress and prove denial. If containment cannot be enabled, display and log “application-level guard only”.

Implementation touchpoints: scan/zbtriage/netguard.py; cli.py; modules/host.py; modules/traces.py

SEC-002 — Privilege and executable boundary

Priority: P0 | Proposed owner: Security/platform

Required implementation: Enforce standard-user startup; no sudo/runas/UAC request by default. Enumerate all subprocess calls centrally: absolute executable path, non-shell invocation, fixed template arguments, closed inherited handles, scrubbed environment, constrained working directory, byte/elapsed limits and return-code inspection. Reject executables under writable search paths and untrusted PATH resolution.

Verification / acceptance: Hostile PATH injection, executable substitution and argument-metacharacter tests produce no arbitrary execution; privileges do not increase; child trees are terminated after timeout.

Implementation touchpoints: scan/zbtriage/modules/host.py; modules/traces.py; phoneguide.py; launch scripts

SEC-003 — Android collector quarantine and independent attestation

Priority: P0 | Proposed owner: Mobile acquisition

Required implementation: Move downloads/launch to explicit acquisition process. Hard-coded SHA-256 alone is insufficient: require publisher signing identity or independently controlled trusted-signature manifest plus exact version/digest and auditable source correspondence; maintain separate review of AndroidQF. Pin approved platform builds; reject unknown architectures, redirects outside allow-list, unexpected files and checksum mismatch; no auto-run after download. Show device-access permissions before consent.

Verification / acceptance: Tampered/unsigned/unapproved binaries fail closed; consent applies to a named digest and mode; dynamic sandbox logs and collector source/binary review artefacts are attached to release evidence.

Implementation touchpoints: scan/zbtriage/phoneguide.py; modules/android.py; README.md

SEC-004 — Private outputs and default minimisation

Priority: P0 | Proposed owner: Privacy/reporting

Required implementation: Default report carries indicators, severity, category and non-sensitive summaries; pseudonymise account names and truncate paths; suppress cookie values, SMS/message text, tokens and unneeded device identifiers. Full artefact inclusion requires separate affirmative consent and recipient encryption. Warn/reject known synchronised/network output locations in protected modes.

Verification / acceptance: Canary strings planted in cookies/messages/credentials do not appear in default JSON/HTML/text reports, logs or error output. Private-file mode verified on each OS.

Implementation touchpoints: scan/zbtriage/report.py; evidence.py; PRIVACY.md

SEC-005 — Scratch lifecycle and decrypted mobile backup control

Priority: P0 | Proposed owner: Privacy/storage

Required implementation: Create one unpredictable local per-run directory with restrictive ownership/ACL; fail if destination is symlink, reparse point, remote or group-writable. Stage SQLite snapshots and decrypted mobile databases there; prefer extracting only necessary rows/columns. Maintain cleanup ledger, recover orphaned runs on next start, and remove on all exception/interrupt paths. Do not promise secure erasure on SSDs.

Verification / acceptance: Inject SIGINT/termination at each extraction phase; verify no orphaned plaintext remains after restart reconciliation. Run as another local user and verify inability to list/read scratch.

Implementation touchpoints: scan/zbtriage/modules/browser.py; ios_sources.py; backupcrypto.py

SEC-006 — Evidence confidentiality, integrity and custody

Priority: P0 | Proposed owner: Evidence

Required implementation: Separate minimally redacted reports from optional full evidence. Use modern authenticated encryption for full exports, generated random data-encryption key, recipient-key wrapping or password KDF with suitable parameters and no secret in command-line arguments. Manifest lists schema, source identity, extraction time (UTC), hashes, tool/build/feed digests, consent and chain-of-custody events. Signature of manifest verifies producer identity but does not replace encryption.

Verification / acceptance: Round-trip encrypted export with wrong-key rejection and tamper detection; plaintext ZIP blocked in protected mode; deterministic manifest validation and independent verify command.

Implementation touchpoints: scan/zbtriage/evidence.py; report.py; handoff.py

SEC-007 — Path traversal, symlink and mount race resistance

Priority: P0 | Proposed owner: Filesystem

Required implementation: Implement handle-relative opens and no-follow checks where possible; bind validated root to file identity and revalidate under race. Disallow extraction entries containing absolute paths, traversal, device names, ADS and dangerous link types; prohibit output/input aliasing and remote paths for sensitive data. Windows reparse/junction logic and POSIX mount cases require separate adapters.

Verification / acceptance: Concurrent symlink/junction swap tests, crafted archives and casefold/Unicode aliases never read outside allow-listed roots or write outside scratch/output. Unsupported safe-open must fail closed in high-assurance mode.

Implementation touchpoints: scan/zbtriage/localpaths.py; readers.py; ios_sources.py; evidence.py

SEC-008 — Bound resource consumption and parser isolation

Priority: P0 | Proposed owner: Runtime/platform

Required implementation: Set documented per-module timeouts, total file count, recursion depth, decompressed bytes, expansion ratio, SQLite query time, individual record size, pattern-search budget, subprocess output cap and total scratch quota. Parse untrusted backup/archive data in worker processes with memory/CPU ceilings where supported. Avoid loading entire multi-GB artefacts.

Verification / acceptance: Oversize/malformed corpus returns bounded “incomplete/resource limit” results without crash, path escape, unbounded memory/disk or silent omission; module-level abort does not corrupt overall report.

Implementation touchpoints: scan/zbtriage/readers.py; modules/patternscan.py; modules/browser.py; modules/ios.py

SEC-009 — Supply-chain and build provenance

Priority: P1 | Proposed owner: Release engineering

Required implementation: Pin production, optional and build dependencies with cryptographic hashes; track Python and OS support; publish CycloneDX or SPDX SBOM; signed tags and release manifest with source ZIP, binary, build inputs and reproducibility claim. Build from protected CI with minimal token permissions and segregated signing keys. Block publishing on critical dependency findings unless documented waiver.

Verification / acceptance: Independent runner reproduces artefact hash where reproducible, otherwise records legitimate differences; every release verifies signatures, SBOM, dependency hashes and security gates.

Implementation touchpoints: requirements-*.txt; build/*; .github/workflows/*

SEC-010 — Verified, rollback-safe IOC feed updates

Priority: P1 | Proposed owner: Indicators/update

Required implementation: Remove ambiguous trust of mutable upstream branches from security-assured use. Accept only signed, versioned feed manifest specifying exact source, SHA-256, record counts, schema and expiry; download to staging, inspect size/content, verify signatures before atomic activation; preserve last known-good set; record update provenance in reports. Network access only in isolated updater.

Verification / acceptance: Corrupt manifest, rollback attempt, digest mismatch, stale expiry or mid-update crash leaves previous feed active; offline mode never fetches.

Implementation touchpoints: scan/zbtriage/feeds.py; iocs/*; build/*

SEC-011 — Explicit external browser-test isolation

Priority: P1 | Proposed owner: UX/security

Required implementation: Move online ad-blocking test outside scan workflows; never navigate without a distinct affirmative user action. Provide full destination, independent ownership/privacy warning and locally served offline alternative. Do not imply a third-party website is a trustworthy malware test.

Verification / acceptance: Normal scan and CLI non-interactive runs never open a browser; UI automation shows no traffic before opt-in.

Implementation touchpoints: scan/zbtriage/menu.py; README.md

SEC-012 — Sensitive logging and exception safety

Priority: P0 | Proposed owner: Privacy

Required implementation: Central structured logger with allow-listed fields and automatic scrubbing; never log decrypted contents, raw URLs containing query secrets, credentials, sensitive snippets or password input. Debug mode requires warning, private storage and short retention. Explicit distinction between recoverable parser errors and trust-boundary violations.

Verification / acceptance: Canary secret scan of stdout/stderr, log files, crash handling and CI outputs finds zero values; errors preserve context without raw private data.

Implementation touchpoints: cli.py; report.py; ios_sources.py; modules/*

SEC-013 — Deterministic scope, consent and no-surprise operations

Priority: P1 | Proposed owner: Core/UX

Required implementation: Capture a user-readable preflight plan listing roots, collectors, outputs, temporary storage, outbound destinations and destructive operations (ideally none). Reject implicit background updates, elevation, auto-open or persistence. Serialize consent decisions, including rejection, without storing personal secrets.

Verification / acceptance: Approval matrix tests show no privileged/network/evidence action can be triggered from scan defaults or a menu shortcut without specific consent.

Implementation touchpoints: scan/zbtriage/cli.py; menu.py; workflow.py; phoneguide.py

SEC-014 — Detection-quality and completeness model

Priority: P1 | Proposed owner: Detection science

Required implementation: Every module emits attempted/succeeded/denied/unsupported/corrupt/resource-limited counts; use explicit unknown state rather than translating unreadable content into negative. Show IOC provenance, feed age, confidence and alternative benign explanations. Separate malicious match, suspicious heuristic and security posture observations.

Verification / acceptance: Fixtures prove missed/permission-blocked sources change coverage, not findings count; output never says “device safe”; precision/recall benchmarks published against representative datasets.

Implementation touchpoints: scan/zbtriage/result.py; contract.py; report.py; modules/*

SEC-015 — Formal report schema and safe rendering

Priority: P1 | Proposed owner: Reporting

Required implementation: Version report JSON schema and escaping rules. Encode HTML without untrusted markup/script injection, protect against spreadsheet formula injection on CSV export and reject unsafe links. Add machine-readable statuses CLEAN_WITHIN_SCOPE, INDICATORS_FOUND, INCOMPLETE, ERROR and CANCELLED (names illustrative).

Verification / acceptance: XSS payloads in URLs/filenames render as plain text; CSV payloads do not run formulas; schema contract and consumer backward-compatibility tests pass.

Implementation touchpoints: scan/zbtriage/report.py; contract.py

SEC-016 — Platform compatibility and execution containment

Priority: P1 | Proposed owner: Platform owners

Required implementation: Maintain supported OS/Python/version matrix; macOS TCC and FDA permission semantics; Windows ACL, Defender and junction semantics; Linux mount, namespace and distro differences. In mobile reports explicitly distinguish host backup analysis from live device inspection.

Verification / acceptance: CI and manual physical-device matrix includes positive and inaccessible cases; each unsupported probe produces explicit explanation and no privilege escalation.

Implementation touchpoints: scan/zbtriage/modules/posix.py; modules/host.py; modules/ios.py; modules/android.py

SEC-017 — Incident handling and authenticity transparency

Priority: P2 | Proposed owner: Maintainers

Required implementation: Publish SECURITY.md reporting policy, signed advisories, release revocation/replacement procedure, verified update keys and threat model. Build reproducible script to verify distribution before running; avoid unqualified “zero breach” claims.

Verification / acceptance: Simulated compromised release and leaked signing key exercises demonstrate incident bulletin, revocation and replacement publication.

Implementation touchpoints: SECURITY.md; README.md; docs/VERIFYING.md

SEC-018 — Performance engineering and graceful cancellation

Priority: P2 | Proposed owner: Performance

Required implementation: Use streaming and indexed queries, cancellation token propagated to workers, bounded parallelism, memory budgets and back-pressure. Expose per-module timing, file count, I/O bytes, peak RSS, scratch peak and reasons for skipped work, without collecting sensitive content.

Verification / acceptance: Benchmark reference datasets and regressions; cancellation cleans children and snapshots; resource ceilings hold on laptops with constrained RAM/disk.

Implementation touchpoints: scan/zbtriage/readers.py; modules/*; workflow.py

4. Concrete API and data contracts

These interfaces are illustrative engineering requirements; exact module names and dataclasses may be refined during implementation while preserving observable behaviour.

```
@dataclass(frozen=True)
class RunPolicy:
    mode: Literal["offline", "offline-high-assurance", "update", "acquire"]
    allowed_roots: tuple[Path, ...]
    network: Literal["deny", "update-allowlist", "collector-disclosed"]
    output: Literal["redacted", "encrypted-evidence"]
    no_elevation: bool = True
    max_total_seconds: int = 3600
    max_scratch_bytes: int = 2_000_000_000
    max_artifact_bytes: int = 500_000_000

@dataclass(frozen=True)
class ModuleResult:
    status: Literal["complete", "partial", "unsupported", "failed"]
    findings: tuple[Finding, ...]
    attempted: int
    inspected: int
    skipped: int
    warnings: tuple[str, ...]
    provenance: dict[str, str]
```

Policy validation precedes all file accesses, process spawns and user prompts. Modules must return structured errors rather than silently continuing. Consent records contain action ID, explicitly disclosed digest/destination, UTC timestamp and approval; no user secrets.

5. Engineering work packages, dependencies and sequencing

Sprint / gate	Work packages	Completion evidence
Gate 0 — containment	SEC-001/002/003; disable unapproved Android path; document truthful platform scope	Threat model, adversarial egress/process tests, separate acquisition review
Gate 1 — privacy	SEC-004/005/006/012; private scratch, redaction, encryption, cleanup	PII canary and interrupted-run reports, export tamper tests
Gate 2 — hostile input	SEC-007/008; safe filesystem primitives, bounded parsers	Fuzz corpus and path-swap proof, peak resource profiles
Gate 3 — authenticity	SEC-009/010/011/013; signatures, updater, consent, browser separation	Verified release, rollback tests, permission approvals
Gate 4 — assurance	SEC-014/015/016/017/018; detection and platform qualification	Precision/recall report, CI matrix, operational release checklist

5.1 Definition of done for every change

- Code change and threat-model update accompany a unique SEC identifier; security-sensitive defaults remain backwards-compatible only where safe.
- Unit tests plus negative/adversarial tests are mandatory; explicitly identify tests needing native OS permissions or physical devices.
- Document any residual risk, platform dependency and user-visible change; do not silently degrade security controls.
- Two independent code reviews for filesystem, crypto, update, subprocess and acquisition changes; second reviewer should not be author.

- CI must reject accidental egress, secrets in logs, dependency issues and broken report schema where those checks are runnable.

6. Security test specification and adversarial matrix

Test ID / threat	Method	Pass criterion
T-NET-01 / Python bypass	socket, urllib, raw socket and DNS while scanner operates	No egress; violation logged without contacting IOC
T-NET-02 / child egress	Spawn permitted fake child attempting DNS/HTTPS	OS containment denies; no successful packets
T-PROC-01 / command injection	Hostile PATH, metacharacters, whitespace, shell metacharacter filenames	Only expected binary and static command run
T-MOB-01 / collector tamper	Corrupt digest, wrong signature, wrong architecture, substituted download	Refused before execute; actionable diagnostics
T-FS-01 / swap attacks	Race symlinks, junctions and root directory rename	No out-of-root reads or writes
T-ARC-01 / decompression bomb	Nested archive and 1:100000 expansion artefacts	Quota triggers, clean partial result
T-DB-01 / hostile SQLite	Malformed headers/WAL/large rows/locked database	Bounded processing, no raw data leakage
T-PRIV-01 / exfiltration canaries	Inject synthetic cookie, token, message and username	None in default exports, debug or stderr
T-PRIV-02 / interruption	Kill at each scratch-write point, restart scan	Clean stale plaintext, ledger complete
T-CRYP-01 / integrity	Flip bit in encrypted evidence, wrong recipient key	Authentication fails; no partial disclosure
T-UPD-01 / supply-chain	Expired/unsigned/rolled-back manifest and partial download	No activation, last known-good retained
T-REP-01 / XSS and CSV	Payloads in artefact names and suspicious URLs	Never executed when reports opened
T-DET-01 / false negatives	Permission denial and deliberately missing source	Incomplete/unknown reported, not clean
T-PERF-01 / stress	Large realistic backups and constrained memory machines	Meets published ceilings; cancellation recovers

Coverage targets are release gates, not arbitrary assurances: all P0 security tests pass on real Windows/macOS/Linux environments; 100% of documented trust-boundary negative cases pass; any known critical exploitable vulnerability blocks release. Detection recall/precision and performance budgets must be set from measured datasets and published alongside hardware profiles rather than invented here.

7. Operational security and performance benchmarks

Collect per-run elapsed time, CPU time, peak resident memory, number of artefacts visited/skipped, scratch peak and byte counts, module errors and feed age. Benchmark on a small clean host, realistic high-volume browsing profile, large iOS backup, Android acquisition output and hostile corpus. Use median and P95 across repeated runs; compare with a frozen baseline. Keep test data synthetic or properly consented and redact identifiers.

Set resource ceilings per host profile at preflight; show predicted scratch capacity and abort safely when free space falls below reserve. All limits are configurable within safe bounds, recorded in report and tested for overflow and integer-edge conditions.

8. Release gate and go/no-go checklist

- Signed source and artefact digests independently verified; SBOM published and dependencies reviewed.

- No unsigned or unapproved third-party collector executing; Android feature disabled where attestation cannot be established.
- Offline scans tested for zero network activity, including child processes; OS enforcement explicitly differentiated from Python-level guard.
- Standard-user, no-elevation and no-persistence behaviour demonstrated on supported operating systems.
- Default reports redacted and full evidence recipient-encrypted; leakage, archive and snapshot interruption tests passed.
- Filesystem traversal, parser fuzz, resource-exhaustion and report-injection tests passed.
- Completeness/error status and limitations accurately shown; mobile platform claims reviewed.
- Independent review signs off P0/P1 issues; remaining findings enter a visible risk register with named accountable owner.

9. Prior audit findings to remediation traceability

Prior finding	Mapped specification	Required closure
F-01 Android downloaded collector	SEC-002, 003, 013	External collector attested and gated
F-02 Sensitive local output	SEC-004, 006, 012	Minimal reports and encrypted custody
F-03 Temporary plaintext	SEC-005, 008	Private scratch and interruption recovery
F-04 Partial network guard	SEC-001, 002	Real OS egress denial evidence
F-05 External browser site	SEC-011, 013	Opt-in separate online operation
F-06 PowerShell boundary	SEC-002, 016	Fixed/absolute bounded subprocess
F-07 Feed supply-chain	SEC-009, 010	Signed immutable feed activation
F-08 False reassurance	SEC-014, 015	Explicit coverage and result limits
F-09 Local path races	SEC-007	Handle-relative defensive file access
F-10 Evidence integrity	SEC-006	Encrypted and authenticated custody
F-11 Dependency risks	SEC-009, 017	SBOM, CI and advisory policy
F-12 Resource exhaustion	SEC-008, 018	Quotas, isolation and benchmarks

10. Implementation map and first code changes

Recommended first pull requests: PR-01 introduce RunPolicy and isolate offline/core/updater/collector command entry points; PR-02 centralise subprocess launching and executable trust checks; PR-03 add secure scratch manager and privacy-safe report defaults; PR-04 add OS network isolation adapters and test harness; PR-05 disable Android execution pending attestation and add signed collector allow-list; PR-06 implement resource and path safety wrappers; PR-07 add signed feed manifest and release SBOM; PR-08 implement coverage ledger and tested results schema.

Each pull request should include migration notes, tests, source audit diff and a proof that the change does not inadvertently introduce greater operating-system privileges or networking than before.

11. Open design decisions and residual risk

- Choose a documented OS-level isolation strategy per platform; not every host permits firewall manipulation by a standard user. High-assurance mode must refuse to claim enforcement when unable to provide it.
- Choose encryption formats, recipient identity and key-recovery UX with cryptographic review; never invent bespoke encryption.
- Determine whether AndroidQF publisher signatures and reproducible builds exist; until established the collector cannot receive trusted release status.
- Decide on supported OS versions and Python releases after testing actual operating systems and mobile acquisition outputs.

- File-system secure deletion cannot reliably be guaranteed on modern flash, snapshots or cloud backups; prioritise encryption and minimisation.
- Neither a green scan nor passing scanner security tests proves absence of kernel compromise or advanced spyware; recommend independent forensic escalation for serious suspicions.

12. Evidence and source references

Source of baseline: Zero-Breach-Toolkit-main.zip (SHA-256 a9b7689137397250f6a9469f3b27369ac96d0f9d10830ed7084982bcced12f3d), assessed 29 October 2026. Existing audit: Zero_Breach_Toolkit_Security_Audit_2026-10-29.docx. Source areas inspected/used for traceability: PRIVACY.md, SECURITY.md, README.md, scan/zbtriage/netguard.py, phoneguide.py, feeds.py, evidence.py, report.py, localpaths.py, cli.py, menu.py, ios_sources.py, readers.py, modules/browser.py, modules/host.py, modules/traces.py, modules/patternscan.py, requirements-*.txt, build/, .github/workflows/ and scan/tests/. These are path-level implementation anchors; the complete implementation still requires a new code review and real-world validation.

Technical standards to consult during engineering: NIST SP 800-218 (SSDF), NIST SP 800-53 (least privilege/audit controls), OWASP ASVS for output/secret-handling principles, OWASP Software Component Verification Standard, SLSA build provenance, SPDX/CycloneDX SBOM specifications and platform vendor documentation for filesystem, process and network sandbox APIs. Standards are guidance, not evidence of present compliance.